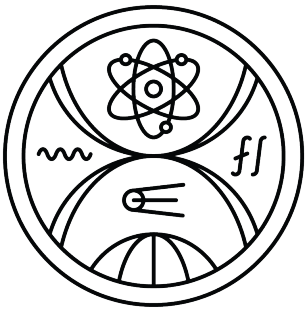
COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

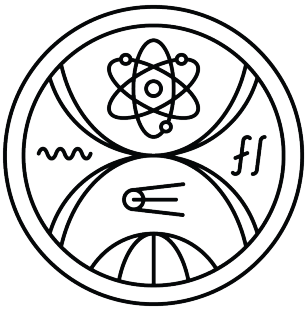


A MODULAR CLUSTER FOR AI RESEARCH IN RESOURCE-CONSTRAINED ACADEMIC ENVIRONMENTS

Master thesis

2026 Bc. Filip Sršeň

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS



A MODULAR CLUSTER FOR AI RESEARCH IN RESOURCE-CONSTRAINED ACADEMIC ENVIRONMENTS

Master thesis

Study program:	Applied informatics
Branch of study:	Applied informatics
Department:	Department of Applied Informatics
Supervisor:	Mgr. Ján Kľuka, PhD.

Bratislava, 2026 Bc. Filip Sršeň

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Filip Sršeň
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: A Modular Cluster for AI Research in Resource-Constrained Academic Environments
Modulárny klaster pre výskum v AI v akademickom prostredí s obmedzenými zdrojmi

Anotácia: Významná časť výskumu na našej katedre sa opiera o metódy strojového učenia, no dostupná výpočtová infraštruktúra je obmedzená a náročná na správu. Výskumníci pravidelne narážajú na problémy so zdieľaním hardvérových zdrojov, efektívnou prácou s dátovými súbormi a presunom experimentov medzi rôznymi prostrediami. Súčasné postupy – ako manuálne plánovanie, udržiavanie samostatných softvérových inštalácií či kopírovanie dát medzi servermi – komplikujú prácu a znižujú jej produktivitu.

Táto práca sa zameriava na riešenie týchto výziev skúmaním možností návrhu modulárneho klastra prispôsobeného potrebám katedry využitím súčasných technológií na virtualizáciu, kontajnerizáciu a správu klastrov implementovaných ako slobodný softvér/softvér s otvoreným zdrojovým kódom (Proxmox, Docker, Kubernetes, SLURM a pod.). Zámerom je navrhnúť riešenie zlepšujúce prístupnosť, škálovateľnosť a efektivitu práce s ohľadom praktické obmedzenia dostupných zdrojov a pracovné postupy používateľov. Implementácia má demonštrovať, ako tieto môžu uvedené technológie podporovať akademický výskum poskytovaním základu, ktorý uľahčuje spoluprácu, zjednodušuje pracovné postupy a zvyšuje reprodukovateľnosť experimentov strojového učenia.

Cieľ: Hlavným cieľom tejto práce je navrhnúť a implementovať modulárny výpočtový klaster prispôsobený technickým a prevádzkovým obmedzeniam akademického výskumného prostredia. Okrem zabezpečenia zdieľanej výpočtovej kapacity bude klaster slúžiť ako platforma integrujúca do jedného celku plánovanie výpočtových úloh, správu prostredí a sledovanie experimentov. Implementácia systému ukáže, ako možno moderné orchestračné technológie prispôsobiť menšej akademickej infraštruktúre s cieľom zlepšiť reprodukovateľnosť a spoľahlivosť, zjednodušiť údržbu a efektívne využívať dostupné výpočtové zdroje.

Literatúra:
Yoo, A.B., Jette, M.A., Grondona, M (2003). SLURM: Simple Linux Utility for Resource Management. In: JSSPP 2003. Springer.
Domingus, J., Arundel, J. (2022): Cloud Native DevOps with Kubernetes, 2nd Edition. O'Reilly.

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Salatino, M. (2024): Platform Engineering on Kubernetes. Manning.
Spišaková, V., Klusáček, D., Hejtmánek, L. (2023). Using Kubernetes in Academic Environment: Problems and Approaches. In: JSSPP 2022. LNCS 13592. Springer
Tim Wickberg (2025). Slinky: Slurm In Kubernetes, Performant AI And HPC Workload Management. In: KubeCon Europe.
Petrosyan, D.A. (2025). Scheduling ML and HPC Jobs with Shoc Platform over Kubernetes. Phys. Part. Nuclei 56, 1581–1585.
Chazapis, A., Nikolaidis, F., Marazakis, M., Bilas, A. (2023). Running Kubernetes Workloads on HPC. In: ISC High Performance 2023. LNCS 13999. Springer.
Misale, C. et al. (2022). Towards Standard Kubernetes Scheduling Interfaces for Converged Computing. In: SMC 2021. CCIS 1512. Springer.

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 31.10.2025

Dátum schválenia: 11.11.2025 prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

I hereby solemnly declare that I have written the entire Master thesison the topic of “A Modular Cluster for AI Research in Resource-Constrained Academic Environments”, including all its appendices and images, independently, using the literature listed in the attached bibliography and artificial intelligence tools. I declare that I have used artificial intelligence tools in accordance with relevant legal regulations, academic rights and freedoms, and ethical and moral principles, while maintaining academic integrity, and that their use is appropriately cited within the work. Specifically, artificial intelligence tools were used for proofreading and correction, comprehensive research, and documentation consolidation.

. . . . . .

Bratislava, 2026 Bc. Filip Sršeň

Acknowledgement

TODO

Abstract

Small academic departments conducting machine learning research face a persistent infrastructure challenge. They cannot afford the dual-stack approach common in larger institutions, i.e., a traditional HPC scheduler such as Slurm for batch training jobs alongside a separate Kubernetes deployment for interactive services, yet operating without any orchestration leads to inefficient resource utilization, manual scheduling conflicts, and poor environment reproducibility. This thesis presents the design and implementation of a modular, Kubernetes-native compute cluster that consolidates batch workloads, interactive development environments, and supporting services onto a single lightweight control plane, purpose-built for resource-constrained academic settings.

The platform is built on K3s, a lightweight Kubernetes distribution whose minimal resource footprint makes a production-grade control plane feasible on a small cluster of consumer-grade machines. An intelligent scheduling layer based on Volcano provides queue-based fair sharing, gang scheduling for distributed jobs, and priority-driven preemption, compensating for the absence of hardware-level GPU partitioning (NVIDIA MIG) on consumer-grade GPUs. Complementary components include JupyterHub for interactive notebook access, Kube-Ray for distributed training, MLflow for experiment tracking, and a private container registry for reproducible environments.

The entire stack is designed for single-administrator operability: node bootstrapping is automated via Ansible, application lifecycle is managed through Helm charts deployed via CI/CD pipelines, and all configuration is version-controlled. The architecture is evaluated on a multi-node cluster of heterogeneous consumer-grade hardware (NVIDIA Titan and GeForce RTX GPUs, 12–24 GB VRAM), demonstrating that a single, unified control plane can effectively serve the diverse workload requirements of an academic AI research lab without dedicated DevOps staff.

Keywords: Kubernetes, GPU cluster management, resource-constrained computing, academic computing infrastructure, container orchestration, batch scheduling

Abstrakt

Malé akademické oddelenia vykonávajúce výskum v oblasti strojového učenia čelia pretrvávajúcej infraštruktúrnej výzve. Nemôžu si dovoliť prístup s dvoma oddelenými systémami, bežný vo väčších inštitúciách, t. j. tradičný HPC plánovač ako Slurm pre dávkové tréningové úlohy popri samostatnom Kubernetes nasadení pre interaktívne služby, no prevádzka bez akejkoľvek orchestrácie vedie k neefektívnemu využívaniu zdrojov, manuálnym konfliktom v plánovaní a nízkej reprodukovateľnosti prostredí. Táto diplomová práca predstavuje návrh a implementáciu modulárneho Kubernetes-natívneho výpočtového klastra, ktorý konsoliduje dávkové úlohy, interaktívne vývojové prostredia a podporné služby na jedinom odľahčenom riadiacom uzle, navrhnutom pre akademické prostredia s obmedzenými zdrojmi.

Platforma je postavená na K3s, odľahčenej distribúcii Kubernetes, ktorej minimálna spotreba zdrojov umožňuje prevádzkovať produkčný riadiaci uzol na malom klastri spotrebiteľského hardvéru. Inteligentná vrstva plánovania založená na Volcano poskytuje spravodlivé zdieľanie zdrojov prostredníctvom front, gang scheduling pre distribuované úlohy a preempciu na základe priorít, čím kompenzuje absenciu hardvérového delenia GPU (NVIDIA MIG) na spotrebiteľských grafických kartách. Medzi doplnkové komponenty patrí JupyterHub pre interaktívny prístup k notebookom, KubeRay pre distribuovaný tréning, MLflow pre sledovanie experimentov a súkromný register kontajnerov pre reprodukovateľné prostredia.

Celý stack je navrhnutý pre prevádzku jediným správcom: bootstrap uzlov je automatizovaný prostredníctvom Ansible, životný cyklus aplikácií je spravovaný cez Helm chart-y nasadzované prostredníctvom CI/CD pipeline a celá konfigurácia je verzionovaná. Architektúra je evaluovaná na viac-uzlovom klastri heterogénneho spotrebiteľského hardvéru (NVIDIA Titan a GeForce RTX GPU, 12–24 GB VRAM), čím sa demonštruje, že jediný zjednotujúci riadiaci uzol môže efektívne obslúžiť rôznorodé požiadavky na pracovné zaťaženie akademického laboratória pre výskum AI bez potreby dedikovaného DevOps tímu.

Kľúčové slová: Kubernetes, správa GPU klastra, výpočty s obmedzenými zdrojmi, akademická výpočtová infraštruktúra, orchestrácia kontajnerov, dávkové plánovanie

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The rapid growth of machine learning model complexity from large language models to diffusion-based generative architectures has created a widening gap between the compute resources available to industry and those accessible to academic researchers. While large technology companies operate dedicated GPU clusters with thousands of accelerators, university departments typically manage a handful of machines with far more modest capabilities. This disparity forces academic AI research groups to either compete for shared institutional HPC resources with long queuing times, or to maintain their own small-scale infrastructure with limited administrative support.

A common response to this challenge in mid-size research institutions is to adopt two separate infrastructure stacks: a traditional HPC scheduler such as Slurm for batch training jobs, and a cloud-native platform such as Kubernetes for long-running interactive services like JupyterHub and experiment tracking dashboards. This *infrastructure sprawl* introduces significant operational overhead, as both stacks require dedicated DevOps expertise to deploy, configure, and maintain. For small academic departments that cannot afford a dedicated infrastructure team, this dual-stack approach is often impractical, leaving researchers to fall back on ad-hoc, unmanaged access to individual machines.

1.1.1 Computing at our department

On a global scale, Comenius University is a rather small institution, with our faculty being an even smaller one, let alone our department. The Department of Applied Informatics has around

TODO: Insert exact number of faculty members.

faculty members and around

TODO: Insert exact number of students.

students. Many of these students are actively doing research in machine learning and adjacent fields, thus requiring computational resources that often go beyond their own personal computers.

The department operates a heterogeneous collection of compute nodes built from consumer-grade hardware. The machines are equipped with a mix of GPUs spanning several generations—including NVIDIA Titan V, Titan XP, GeForce RTX 3060, and GeForce RTX 4090 with VRAM capacities ranging from 12 GB to 24 GB. CPU configurations and RAM sizes similarly vary across nodes. This hardware heterogeneity, while cost-effective, presents its own set of challenges for workload placement and resource management. On request, these machines are available to any student who needs them for their work.

1.1.2 Current state

Currently there are four machines used for compute. Users are provided access to each machine the department manages based on a request from their supervisors. They are also added to a Teams team, which is used to post updates about the machines. It is also used for "reserving" compute slots users post to the team when, for how long, and on which machine they are going to be computing. Users then use SSH to connect to the selected nodes, move their data to the node, and run their computations.

However, because the computers are standalone machines with no collective management or shared file system beyond user logins, this setup comes with a number of challenges: environment management, data management, and scheduling. These are discussed in detail in the problem statement.

This ad-hoc arrangement illustrates the broader problem facing small research groups: without a unified platform, departments must choose between operating without any orchestration accepting the inefficiencies described above or adopting heavyweight infrastructure stacks that require dedicated administrators. Neither extreme serves the needs of a small academic lab well. The solution explored in this thesis seeks a middle ground: a *single*, lightweight control plane that consolidates batch workloads, interactive development environments, and supporting services onto one platform manageable by a single administrator.

1.2 Problem statement

The complexity of managing distributed AI workloads and the difficulty of providing a "single-pane-of-glass" interface for researchers without heavy DevOps overhead.

1.2.1 Data and storage management

Machine learning often requires large datasets that need to be present for training and evaluating models. Users transfer their datasets onto the machines manually ahead of computation. The datasets then reside on the machines indefinitely. Two problems stem from this:

- When the machine a user has all their data on is being used by someone else, the user needs to either wait for the other job to finish or move the data elsewhere, causing further strain on the machine.

- Users are often not mindful about deleting their unused data. This causes the file system to fill up and requires periodical clean-ups by the administrator.

1.2.2 Environment management

Users manage their environments, packages, and libraries through Conda. As with data, however, these environments are not shared across the machines. This means that whenever users want to switch machines they need to take their Conda environments with them. The result is that the same packages are installed multiple times across the machines, increasing file system utilization further.

TODO: Quantify the storage waste if possible — e.g., typical Conda environment size multiplied by number of duplicate installations observed across machines.

1.2.3 Scheduling

Whenever users want to use the machines for compute they are supposed to post into a Teams channel. The post should contain information about their job, mainly the length of the computation and the machine(s) they are selecting. Many times, however, the time estimate is inaccurate, or even worse users do not post about their job in the channel and just start using the machines. This results in users needing to check each machine individually to confirm whether it is available. There is no centralized monitoring solution, so users manually connect to each node and check its utilization.

TODO: Add a concluding paragraph that transitions to the contribution/design chapters — summarise how the three problems (data, environment, scheduling) motivate the three-part contribution (research, design, implementation).

1.3 Thesis Contributions

This work has three main goals: research, design, and implementation. Together, they address the infrastructure sprawl problem identified in the motivation: small academic departments need a *single*, unified platform that is both feature-rich enough to support modern AI/ML workflows and lightweight enough to be operated by a single administrator on consumer-grade hardware.

1.3.1 Research

The research goal is to conduct an analysis of compute resource usage at our department. This includes describing different use-cases users have for the system in place and creating typical usage scenarios for both users and administrators. User interviews were conducted with students experienced with the current solution, covering their workflows, data, experiment outputs, hardware and software needs, limiting factors, and pain points.

TODO: Forward-reference the chapter/section where interview findings are presented in detail.

1.3.2 Design

The design goal is to design solutions that support the identified use-cases. The solution should:

- be able to exist within the existing infrastructure,
- be easily scalable,
- provide additional services,
- support modern AI/ML practices,
- be maintainable long-term by a **single administrator**, thereby minimizing the total cost of ownership,
- **consolidate batch and interactive workloads** onto a single control plane, eliminating the need for separate HPC and cloud-native stacks,
- operate efficiently on **consumer-grade hardware** without relying on enterprise features such as NVIDIA MIG for GPU partitioning, InfiniBand for high-speed interconnects, or ECC memory.

1.3.3 Implementation

After providing solutions that address users' needs, the goal is to select a solution and implement it. Specifically, this thesis contributes:

- A modular, Kubernetes-native platform for AI research.
- An intelligent scheduling layer for resource-efficient batch processing.
- A distributed executor for scaling research workloads.
- A converged, lightweight control plane, built on K3s, that unifies batch scheduling, interactive development environments, and experiment tracking with minimal operational overhead, designed to be managed by a single administrator on consumer-grade hardware.

1.4 Thesis Structure

Overview of the subsequent chapters.

TODO: Add structure overview

Chapter 2

Background & Related Work

2.1 Cloud-Native Orchestration

This section introduces the foundational concepts behind cloud-native computing and container orchestration that underpin the system designed in this work. We trace the evolution from monolithic deployments through virtualization to modern container-based platforms, with a focus on the technologies and abstractions that enable efficient resource sharing in multi-user environments.

2.1.1 Containerization Fundamentals

Containerization is an operating-system-level virtualization technique that isolates applications and their dependencies into lightweight, portable units called *containers*. Unlike traditional hardware virtualization, which requires a full guest operating system per virtual machine, containers share the host kernel and use Linux kernel primitives, namely *namespaces* for isolation and *cgroups* for resource limiting to achieve confinement with minimal overhead [Sultan et al., 2019].

The Docker project, released in 2013, popularized container technology by providing a streamlined workflow for building, distributing, and running container images. Docker introduced a layered filesystem model and a declarative image format that encodes the full dependency stack of an application, ensuring reproducibility across environments. This portability addressed a persistent pain point in scientific computing, where researchers frequently struggled to reproduce experiments due to differences in library versions, system packages, and hardware configurations [Domingus et al., 2022].

The Open Container Initiative (OCI), established in 2015 under the Linux Foundation, standardized the container image format and runtime specification, decoupling the ecosystem from any single vendor. Today, multiple runtimes (runc, crun, containerd, CRI-O) comply with the OCI specification, and registries serve as universal

distribution channels for container images.

For academic computing, containers offer three key advantages over traditional deployment models. First, *reproducibility*: an experiment packaged as a container image produces identical results regardless of the underlying host, eliminating the "works on my machine" problem. Second, *isolation*: cgroups and namespaces prevent workloads from interfering with one another, enabling safe multi-tenancy on shared infrastructure. Third, *portability*: the same container image can run on a researcher's laptop, a department server, or a cloud provider without modification.

Figure 2.1 illustrates the architectural difference between the two approaches: a hypervisor virtualizes physical hardware and runs a full guest operating system in every virtual machine, whereas a container engine shares the host kernel and isolates processes through namespaces and cgroups, eliminating the per-VM operating system overhead [Sultan et al., 2019].

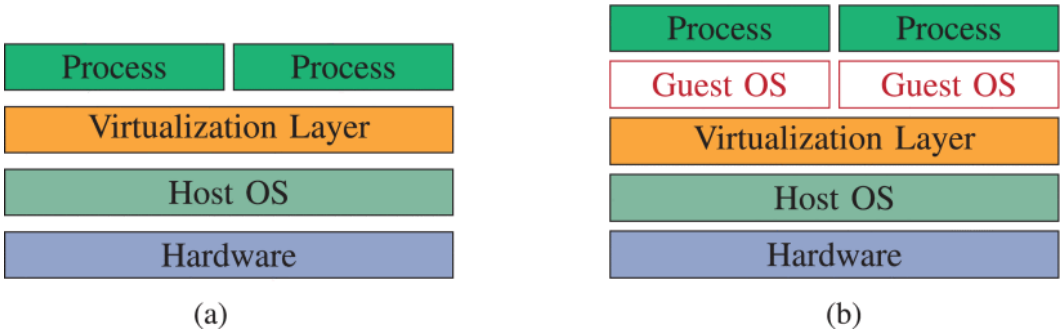


Figure 2.1: Comparison of container architectures (a) and virtual machine (b) . Virtual machines each run a full guest operating system on top of a hypervisor, while containers share the host kernel and are isolated by the container engine. Adapted from Sultan et al. [2019].

2.1.2 Container Orchestration Platforms

As container adoption grew beyond single-host deployments, the need for automated management of container lifecycles across clusters of machines became apparent. Container orchestration platforms address this need by providing centralized control over deployment, scaling, networking, and fault tolerance of containerized workloads.

Google's internal cluster management systems, Borg and Omega, pioneered many of the concepts that underpin modern orchestration [Verma et al., 2015]. Borg, in production at Google since 2008, runs hundreds of thousands of jobs from many thousands of applications across clusters of up to tens of thousands of machines each. It introduced the notions of declarative job specifications, automated placement based on resource constraints, and self-healing through continuous reconciliation of desired and

actual state. Omega extended these ideas with a more modular architecture based on a shared-state scheduling model.

Burns et al. [2016] describe the evolution from Borg through Omega to Kubernetes, an open-source system released by Google in 2014 that has since become the de facto standard for container orchestration. Kubernetes retains Borg's core design principles—declarative configuration, a reconciliation loop, and label-based object selection—while introducing a more extensible API-driven architecture.

The key Kubernetes abstractions relevant to this work include:

- **Pods**, the smallest deployable units, which encapsulate one or more containers sharing network and storage resources on a single node.
- **Deployments** and **StatefulSets**, which manage replica sets of Pods with different guarantees regarding ordering, persistence, and identity.
- **Services**, which provide stable network endpoints for dynamically placed Pods through label selectors.
- **Custom Resource Definitions (CRDs)**, which extend the Kubernetes API with domain-specific objects, enabling the platform to manage workloads beyond traditional microservices.

The Kubernetes control plane consists of several cooperating components: the *API server* exposes the cluster state via a RESTful interface; the *etcd* distributed store persists this state; the *controller manager* reconciles current state with desired state through a collection of control loops; and the *kubelet* agent on each worker node ensures that the containers assigned to it remain running and healthy. This architecture enables both human operators and automated systems to interact with the cluster uniformly through the API.

TODO: Add a diagram of the Kubernetes control plane architecture (API server, etcd, controller manager, scheduler, kubelet, kube-proxy).

2.1.3 Lightweight Kubernetes Distributions

While the standard Kubernetes distribution is well-suited for large-scale data center deployments, its control plane imposes non-trivial resource requirements—etcd, the API server, the controller manager, and the scheduler together consume several gigabytes of memory even on idle clusters. This overhead is acceptable in enterprise settings but becomes a concern in resource-constrained environments such as edge nodes, IoT gateways, and small academic compute clusters [Yakubov and Hästbacka, 2025a].

Several lightweight Kubernetes distributions have emerged to address this gap. K3s, developed by Rancher Labs (now part of SUSE), is a CNCF-certified Kubernetes distribution packaged as a single binary under 100 MB. It replaces etcd with SQLite as the default datastore (with options for external databases), bundles Traefik as the default ingress controller, and strips out legacy alpha features and cloud-provider-specific plugins. The result is a fully conformant Kubernetes control plane with a significantly reduced memory and disk footprint, suitable for nodes with as little as 512 MB of RAM [Yakubov and Hästbacka, 2025a].

Yakubov and Hästbacka [2025a] provide an empirical comparison of five lightweight Kubernetes distributions—K3s, k0s, KubeEdge, OpenYurt, and standard Kubernetes—on resource-constrained edge devices. Their results demonstrate that K3s exhibited the lowest resource consumption across all tested workloads, while matching or exceeding the performance of other distributions under heavy stress scenarios. Yakubov and Hästbacka [2025b] complement this analysis from a security and maintainability perspective, finding that K3s and k0s offer superior ease of development due to their architectural simplicity, though with somewhat lower security compliance compared to full Kubernetes.

For the purposes of this thesis, K3s represents a pragmatic choice: it provides the full Kubernetes API surface and ecosystem compatibility—including support for CRDs, custom controllers, and scheduler plugins such as Volcano—while consuming a fraction of the resources required by a standard cluster. This makes it feasible to run a production-grade control plane alongside compute workloads on a small cluster of consumer-grade machines, managed by a single administrator.

2.1.4 Kubernetes Scheduling and Resource Management

The Kubernetes scheduler is responsible for placing Pods onto nodes that satisfy their resource and constraint requirements. The default scheduler operates in two phases: *filtering*, which eliminates nodes that do not meet hard constraints (resource capacity, node selectors, taints and tolerations), and *scoring*, which ranks the remaining nodes using configurable priority functions [Hightower et al., 2019].

Each container in a Pod specifies *resource requests* (the minimum guaranteed amount) and *limits* (the maximum consumable amount) for CPU and memory. The scheduler uses requests during placement to avoid overcommitting nodes, while limits enforce hard boundaries at runtime via cgroups. Based on the relationship between requests and limits, Kubernetes assigns one of three Quality of Service (QoS) classes to each Pod: *Guaranteed* (requests equal limits), *Burstable* (requests less than limits), and *BestEffort* (no requests or limits specified). Under resource pressure, the kernel's OOM killer preferentially evicts BestEffort and then Burstable Pods, protecting Guaranteed

workloads [Senjab et al., 2023].

For elastic scaling, Kubernetes provides the Horizontal Pod Autoscaler (HPA), which automatically adjusts the number of Pod replicas based on observed metrics such as CPU utilization or custom application metrics [Nguyen et al., 2020]. The Vertical Pod Autoscaler (VPA) complements HPA by adjusting individual container resource requests, and the Cluster Autoscaler dynamically adds or removes nodes to match aggregate resource demand.

TODO: Add a table summarizing resource requests vs. limits and their QoS class implications with concrete examples (CPU/memory values).

The default scheduler is designed for long-running services with relatively homogeneous resource requirements. Batch, scientific, and AI training workloads, however, present distinct scheduling challenges: gang scheduling (all-or-nothing placement of co-dependent tasks), fair sharing among users, preemption of lower-priority jobs, and efficient GPU allocation. To address these needs, the Kubernetes scheduling framework was extended with a plugin architecture that allows custom scheduling logic to be injected at well-defined extension points [Misale et al., 2022]. Projects such as Volcano, Yunikorn, and Kube-Ray leverage this extensibility to provide batch and AI-aware scheduling policies, a topic explored further in Section 2.3.

TODO: Expand on Volcano's gang scheduling and queue management in more detail—this is directly used in the implementation (see orchestration.tex). Reference Volcano-specific literature or docs.

2.1.5 Cloud-Native in Academic and HPC Environments

Traditional high-performance computing (HPC) centers have long relied on batch scheduling systems such as SLURM [Yoo et al., 2003], PBS, or LSF to manage resource allocation. These systems excel at scheduling tightly-coupled parallel jobs on dedicated nodes with strict fairness policies, but they lack native support for the container lifecycle management, service discovery, and declarative APIs that characterize the cloud-native paradigm.

Conversely, Kubernetes was originally designed for stateless microservices and web applications, not for the high-throughput, multi-tenant GPU workloads common in academic research. This mismatch creates a gap that several recent efforts have sought to bridge from both directions.

Spišaková et al. [2023] identify the key problems encountered when deploying Kubernetes in an academic environment, including limited GPU sharing, the absence of fair-share scheduling, and difficulties in providing interactive access alongside batch jobs. Their work proposes approaches for adapting Kubernetes abstractions to aca-

demic workflows while preserving the flexibility and reproducibility benefits of containers.

From the HPC side, Chazapis et al. [2023] explore running unmodified Kubernetes workloads directly on HPC clusters, proposing a design that integrates with the existing job management system rather than replacing it. This avoids partitioning the cluster into separate cloud and HPC partitions and retains established accounting and scheduling policies. The Slinky project [Wickberg, 2025] takes the complementary approach of embedding SLURM's scheduling capabilities within Kubernetes, providing familiar HPC job submission semantics on top of a cloud-native infrastructure.

Misale et al. [2022] propose standard scheduling interfaces for converged computing, arguing that the Kubernetes scheduling framework's plugin architecture can serve as a unifying layer for both cloud-native and HPC workloads. Their work demonstrates how batch scheduling semantics such as gang scheduling, queue management, and job preemption can be implemented as Kubernetes scheduler plugins without modifying the core control plane.

TODO: Add a comparison table: SLURM vs. Kubernetes feature matrix (scheduling model, container support, GPU management, fair-share, API style, multi-tenancy). This table would help motivate the design choices in the architecture chapter.

These convergence efforts are directly relevant to this thesis. The modular cluster designed in this work must serve both interactive research sessions (notebooks, development environments) and long-running batch jobs (model training, hyperparameter sweeps), a duality that requires drawing on techniques from both the cloud-native and HPC traditions. The platform engineering approach of extending Kubernetes through CRDs and custom controllers, rather than modifying core components, provides the architectural foundation for this integration [Salatino, 2024, Domingus et al., 2022].

A recurring pattern in mid-size academic research environments is what we term *infrastructure sprawl*: the coexistence of a traditional HPC scheduler (typically Slurm) for batch training jobs and a separate Kubernetes deployment for long-running services such as JupyterHub, MLflow, or experiment tracking dashboards. Each stack brings its own networking, storage, authentication, and monitoring requirements, effectively doubling the operational surface. Maintaining two independent control planes demands either a dedicated DevOps team, a luxury most small departments cannot afford, or results in one of the two stacks being under-maintained, insecure, or simply abandoned. Lightweight Kubernetes distributions such as K3s offer a path toward consolidation: a single, resource-efficient control plane capable of hosting both batch workloads (via scheduler plugins like Volcano) and interactive services, thereby eliminating the need for a parallel HPC layer. This unification is the core architectural hypothesis explored in this thesis.

TODO: Tie the concluding paragraph more explicitly to the specific design decisions made in this thesis—reference the architecture chapter (e.g. Volcano integration, Kube-Ray for distributed training) once those sections are written.

2.2 Traditional HPC Scheduling vs. Cloud-Native Orchestration

A central design decision for any compute cluster is the choice of workload management paradigm. The two dominant approaches in research computing are traditional HPC schedulers, exemplified by Slurm, and cloud-native container orchestration platforms, exemplified by Kubernetes. This section compares the two paradigms at a general level, highlighting their respective strengths and limitations in the context of AI/ML research workloads.

2.2.1 Traditional HPC: Slurm

Slurm is the de facto standard workload manager in academic and institutional HPC environments. It provides a mature, battle-tested scheduling engine designed for high-throughput batch processing on large clusters.

TODO: Add a citation for Slurm — e.g., the original Slurm paper (Yoo et al., 2003) or SchedMD documentation. Also cite adoption statistics (e.g., TOP500 survey).

Strengths

- **Fair-share scheduling.** Slurm provides sophisticated priority-based scheduling with fair-share policies, allowing administrators to ensure equitable resource distribution across users and groups over time.
- **Efficient resource allocation.** Slurm's job-level resource model (CPU cores, memory, GPUs as discrete allocatable units) is well-suited for tightly-coupled parallel workloads such as MPI jobs, simulations, and large-scale batch processing.
- **Mature ecosystem.** Decades of development have produced a rich toolchain: advanced reservation systems, array jobs, job dependencies, detailed accounting, and extensive administrative tooling.
- **Proven at scale.** Slurm reliably manages clusters ranging from tens to hundreds of thousands of nodes in production environments worldwide.

TODO: Cite the original Slurm paper or a survey of HPC scheduler adoption to back these claims.

Limitations

- **Environment rigidity.** Slurm assumes a shared file system and a uniform software environment across nodes. Users typically rely on environment modules or manual Conda setups, which are not portable across clusters and can lead to version conflicts.

  TODO: Cite a source on reproducibility challenges in HPC environments (e.g., a Singularity/Apptainer paper or an HPC reproducibility study).

- **Limited support for interactive workloads.** While Slurm can allocate interactive sessions, its design is fundamentally batch-oriented. Long-running interactive services such as Jupyter notebooks, experiment tracking dashboards, or web-based development environments do not fit naturally into the job scheduling model.

  TODO: Cite work on interactive HPC or Jupyter-on-Slurm approaches to strengthen this claim.

- **No native container orchestration.** Slurm can launch containers (e.g., via Singularity/Apptainer), but it does not manage container lifecycles, networking, or service discovery. Containerized services must be managed externally.

  TODO: Cite the Singularity/Apptainer paper since it is mentioned by name.

- **Monolithic scheduling model.** Slurm's scheduler treats jobs as opaque resource requests. It has no awareness of application-level semantics such as distributed training topology, gang-scheduling requirements, or service dependencies.

2.2.2 Cloud-Native: Kubernetes

Kubernetes is a container orchestration platform originally developed for microservice workloads, but increasingly adopted for AI/ML and batch processing through extensions such as Volcano and Kubeflow.

TODO: Cite Kubeflow and Volcano project papers or official documentation here.

Strengths

- **Container-native environment management.** Every workload runs in an isolated, reproducible container image. Dependencies, libraries, and runtime versions are bundled with the application, eliminating environment conflicts and enabling portability.

  TODO: Back the reproducibility claim with a citation (e.g., a study on container-based reproducibility in scientific computing).

- **Service-oriented architecture.** Kubernetes natively supports long-running services (JupyterHub, MLflow, Weights & Biases, dashboards) alongside batch workloads, providing a unified platform for both interactive and non-interactive use.

- **Declarative infrastructure.** Cluster state is defined declaratively through manifests and managed via GitOps workflows, enabling reproducible deployments, version-controlled configuration, and easy cluster reconstruction.

- **Rich ecosystem for ML.** Specialized frameworks such as Kubeflow, Volcano (batch scheduling), Kube-Ray (distributed compute), and MLflow integration provide purpose-built tooling for AI/ML workflows including distributed training, hyperparameter tuning, and experiment tracking.

  TODO: Cite Kube-Ray, Kubeflow, and Volcano individually rather than listing them without references.

- **Extensibility.** Kubernetes' operator pattern and custom resource definitions (CRDs) allow the platform to be extended with domain-specific scheduling semantics without modifying the core system.

  TODO: Cite the Kubernetes operator pattern paper or official documentation.

Limitations

- **Operational complexity.** Kubernetes has a steep learning curve and significant operational overhead. Cluster management requires expertise in networking, storage, security, and container registries.

  TODO: Cite a study or blog post quantifying Kubernetes operational complexity.

- **Resource overhead.** The control plane (API server, etcd, scheduler, controller manager) consumes resources that may be significant on small clusters, leaving less headroom for compute workloads.

  TODO: Quantify the control plane resource overhead (e.g., typical memory/CPU footprint of API server + etcd on a small cluster).

- **Weaker fair-share scheduling by default.** Kubernetes' default scheduler lacks the sophisticated fair-share and priority-decay policies that Slurm provides out of the box. These must be added through extensions such as Volcano.

- **Less mature for tightly-coupled parallel jobs.** Kubernetes is not traditionally designed for MPI-style tightly-coupled parallelism. While support exists (e.g., through MPI operators), it is less mature than Slurm's native MPI integration.

  TODO: Cite the MPI Operator project or Kubeflow MPI documentation.

2.2.3 Comparison summary

Neither paradigm is universally superior — the choice depends on workload characteristics and operational requirements. Slurm excels in environments dominated by traditional batch HPC workloads with homogeneous software stacks. Kubernetes excels where workloads are diverse (batch + interactive + services), environments need to be reproducible and portable, and the research community benefits from modern ML tooling.

For AI/ML research environments specifically, the container-native model offers compelling advantages: reproducible environments, native support for interactive development tools, and a growing ecosystem of ML-specific orchestration frameworks. These benefits must be weighed against the operational complexity and resource overhead that Kubernetes introduces, particularly in resource-constrained academic settings.

This thesis adopts Kubernetes as its orchestration platform—specifically, the K3s lightweight distribution discussed in Section 2.1—rather than a traditional HPC scheduler such as Slurm. This choice is driven by three considerations specific to the target environment. First, the workload mix at our department is heterogeneous: users require both long-running interactive sessions (Jupyter notebooks, remote development environments) and batch training jobs, a duality that Kubernetes handles natively while Slurm does not. Second, the operational capacity of the department is limited; a lightweight, single-binary control plane such as K3s can be maintained by a single administrator, whereas deploying and operating Slurm alongside a separate service

platform would effectively double the infrastructure management burden. Third, the Kubernetes ecosystem provides purpose-built extensions for ML workloads—Volcano for batch scheduling, Kube-Ray for distributed training, and MLflow for experiment tracking—that can be integrated incrementally via CRDs without modifying the core platform. The design decisions arising from this choice are detailed in Chapter 3.

2.3 Batch vs. Interactive Computing

The challenge of scheduling high-throughput AI jobs alongside interactive researcher notebooks.

TODO: Expand this into a full introductory paragraph explaining the tension between batch and interactive workloads and why it matters for the platform design.

2.3.1 Common use-cases

Through user interviews with students experienced with the current solution, several common use cases were identified:

TODO: Reference the methodology section or appendix where interview details are described (sample size, interview protocol, etc.).

1. **Script-based batch execution.** Accessing the servers via SSH, copying scripts and datasets onto the machine(s), and running the scripts directly.

2. **Third-party experiment management.** Accessing the servers via SSH, running third-party daemons on the servers, and managing experiments from a web interface. This allows users to run distributed scripts, optimize hyper-parameters of the models, and provides useful metrics about the training process.

   TODO: Name the specific third-party tools mentioned in interviews (e.g., Weights & Biases, MLflow, Optuna) to make the use-case concrete.

3. **Remote development environment.** Using the servers as remote development environments. Users SSH onto the servers and work on their research remotely. This centralization is important for them as they can use thin clients, and if they need to run any quick GPU-heavy calculations to validate their work, there is no need to move to another machine.

TODO: Add subsections expanding on the scheduling challenges for each use-case type and how they conflict (e.g., a long-running notebook holding a GPU vs. a batch training job queuing for it). Forward-reference Volcano's queue/priority mechanisms as the proposed solution.

2.4 Distributed Machine Learning

Paradigms of data and model parallelism and the need for specialized orchestrators.

2.5 State of the Art

Review of existing frameworks (e.g., Kubeflow) and identifying the gap (e.g., complexity, lack of modularity for specialized academic clusters).

TODO: Rewrite this introductory paragraph — the old placeholder text is mixed with the new content. Provide a proper introduction that outlines what the SoTA review covers and its structure.

2.5.1 Technologies in academic practice

There is a common trend of using Python for the majority of development in AI research. Many users work with common machine learning frameworks such as TensorFlow or PyTorch.

TODO: Cite adoption statistics for Python/PyTorch/TensorFlow in ML research (e.g., Stack Overflow survey, Papers with Code statistics).

Some users develop their scripts as Jupyter notebooks. Some convert them into pure Python scripts and run them on the machines. Those who use servers as remote development machines run the notebooks directly.

Users also require metrics from their experiments, whether produced by the ML frameworks themselves or by an ML platform such as Weights & Biases.

TODO: Expand with a brief discussion of experiment management platforms — W&B, MLflow, Neptune — and their relevance to the platform design.

TODO: Add a subsection reviewing existing ML platform frameworks (Kubeflow, Polyaxon, Determined AI, Ray) and identifying their gaps for small academic clusters. This is the core of the SoTA section and is currently missing. Include a comparison table of features relevant to academic use-cases.

2.5.2 Hardware requirements

Computation

Almost all users use the machines to train machine learning models. This usually requires GPUs to accelerate the process. Users also want to utilize multiple GPUs to further speed up the process.

TODO: Cite or reference distributed training paradigms (data parallelism, model parallelism) as motivation for multi-GPU support.

Some users — namely those that work with tabular data — want to run ETL pipelines on the data. This is normally a CPU- and memory-heavy task.

Data storage

A significant portion of research is done in computer vision. This use-case is the most resource-intensive, as image or video datasets can reach hundreds of gigabytes depending on the project.

TODO: Add approximate dataset sizes observed in interviews to ground these claims.

More standard machine learning projects require far less data storage — in the range of megabytes to several gigabytes.

There are also users who work with many small files, which can be quite IOps-intensive.

TODO: Expand the SoTA section significantly: review Kubeflow, Volcano, Kube-Ray, and at least 2–3 competing platforms with citations. Include a comparison table of features relevant to academic use-cases. This section is the backbone for justifying the design choices.

Chapter 3

System Architecture Design

3.1 Introduction

3.1.1 Design Objectives

The architecture is guided by five primary objectives, each addressing a specific aspect of the problem statement and the infrastructure sprawl challenge identified in Chapter 2.

Modularity. The system must be composed of loosely coupled, independently deployable components. This allows individual services: the scheduler, the notebook platform, the experiment tracker to be upgraded, replaced, or removed without affecting the rest of the stack. Modularity also enables incremental adoption: a department can start with a minimal deployment and add components as needs evolve.

Scalability. The architecture must accommodate growth from a single-node proof-of-concept to a multi-node cluster without fundamental redesign. This includes both horizontal scaling (adding compute nodes) and vertical scaling (upgrading individual nodes with better GPUs or more RAM).

Resource Efficiency. Given the limited compute capacity of a small departmental cluster, the system must minimize overhead, both from the control plane itself and from idle or underutilized workloads. Every gigabyte of RAM and every GPU cycle consumed by infrastructure components is unavailable for research.

Single-Administrator Operability. The entire platform from initial deployment through day-to-day operations, monitoring, upgrades, and incident response must be manageable by a single person, typically a PhD student or lab administrator who splits their time between research, teaching and infrastructure maintenance. This

objective shapes every design decision, favoring declarative configuration, automated reconciliation, and a reduced operational surface area. The goal is to minimize the total cost of ownership (TCO) to a level sustainable for a small academic department.

Consumer-Grade Hardware Compatibility. The architecture must operate effectively on consumer-grade hardware without relying on enterprise data center features. Specifically, the system must function without NVIDIA MIG for hardware-level GPU partitioning, without InfiniBand or RDMA for high-speed inter-node communication, and without ECC memory. The design must instead provide software-level mechanisms: scheduling policies, resource quotas, and isolation boundaries, to compensate for the absence of these features.

3.1.2 Design Philosophy

The design follows a cloud-native, microservice-based approach using Kubernetes as the foundational orchestration layer. Rather than building a monolithic platform, each of the following capabilities is realized as an independent service integrated through the Kubernetes API: identity management, notebook provisioning, batch scheduling, experiment tracking, container registry.

A key architectural decision is the selection of *K3s* as the Kubernetes distribution. As discussed in Section 2.1, K3s provides a fully conformant Kubernetes control plane with a significantly reduced resource footprint, packaged as a single binary. For a small academic cluster where control-plane resources compete with research workloads for the same physical machines, this efficiency is essential. K3s also simplifies operational procedures such as cluster bootstrapping, upgrades, and backup, which are handled through a unified toolchain, directly supporting the single-administrator operability objective.

The platform is extended through standard Kubernetes mechanisms: CRDs define domain-specific objects (e.g., Ray clusters, Volcano queues), Helm charts package and parameterize deployments, and a CI/CD pipeline (GitHub Actions) ensures that Helm releases are applied consistently and reproducibly on every configuration change. This approach avoids modifying core Kubernetes components, maintaining compatibility with the broader ecosystem while keeping the operational knowledge transferable.

3.2 High-Level System Model

3.2.1 Layered Abstraction Framework

Identity, Interface, Orchestration, and Lifecycle.

3.2.2 System Interaction Model

End-to-end workflow from auth to distributed training.

3.2.3 The Modular Interconnectivity

Decoupling via standardized interfaces (APIs/K8s CRDs).

3.3 Detailed Component Architecture

3.3.1 Identity and Governance Layer

Keycloak integration for multi-tenancy.

3.3.2 Lightweight Control Plane

The foundation of the architecture is K3s, a lightweight Kubernetes distribution selected for its minimal resource footprint and single-binary deployment model. As detailed in Section 2.1, K3s provides a fully conformant Kubernetes control plane while consuming a fraction of the resources required by a standard deployment. On a cluster of consumer-grade machines, this efficiency is critical: the control plane (API server, scheduler, controller manager) runs on the same physical nodes as research workloads, and every megabyte of RAM devoted to infrastructure reduces the resources available for GPU-accelerated training.

K3s replaces the standard etcd cluster with SQLite as the default embedded datastore for single-server deployments, with options for external databases (PostgreSQL, MySQL, or etcd) for high-availability configurations. For the target environment, a small cluster of four to eight nodes, a single-server deployment with automated backups provides sufficient reliability without the operational complexity of a multi-node etcd quorum.

The deployment lifecycle is managed through a CI/CD pipeline: all cluster configuration, Helm chart values, namespace definitions, and Volcano queue policies is stored in a version-controlled repository. GitHub Actions automates the linting, templating, and application of Helm releases on each push to the main branch. This approach supports the single-administrator operability objective by making all configuration changes auditable, reproducible, and reversible. An administrator can reconstruct the platform from bare metal by running the Ansible playbooks to bootstrap K3s, then pushing the Helm configuration repository to trigger the deployment pipeline in a process that requires no manual intervention beyond the initial node provisioning.

3.3.3 Research Interface and Environment Provisioning

JupyterHub and Harbor (private registry).

3.3.4 Distributed Orchestration and Scheduling Engine

Volcano

Batch scheduling and gang-scheduling for AI workloads. As described in Section 3.4, Volcano's queue management and priority mechanisms are specifically leveraged to compensate for the absence of hardware-level GPU partitioning on consumer-grade cards. By providing software-enforced fair sharing, gang scheduling for distributed jobs, and priority-based preemption, Volcano enables multi-tenant GPU sharing on hardware that would otherwise support only exclusive, single-user allocation.

Kube-Ray

Distributed compute scaling.

3.3.5 Experiment Lifecycle and Metadata Management

MLflow integration for tracking metrics/artifacts.

3.4 Design for Resource Constraint Mitigation

A defining characteristic of the target environment is that all compute nodes are built from consumer-grade hardware. While this keeps procurement costs low, an essential requirement for small academic departments. It introduces a set of constraints that enterprise data center infrastructure is designed to avoid. This section details these constraints and describes the architectural strategies employed to mitigate their impact on system stability, multi-tenancy, and overall research productivity.

3.4.1 Consumer-Grade Hardware Constraints

The department's compute nodes are equipped with a heterogeneous mix of NVIDIA GPUs spanning several generations, with VRAM capacities ranging from 12 GB to 24 GB. This hardware profile imposes three categories of constraints that the architecture must address.

No hardware-level GPU partitioning. Enterprise GPUs such as the NVIDIA A100 and H100 support Multi-Instance GPU (MIG), which allows a single physical GPU to be partitioned into multiple isolated instances with independent memory and

compute units. Consumer-grade GPUs (GeForce and Titan series) do not support MIG. This means that the architecture cannot rely on hardware-enforced isolation between GPU workloads belonging to different users. Instead, isolation must be achieved through scheduling policies and software-level resource quotas.

Limited and heterogeneous VRAM. The available VRAM per GPU ranges from 12 GB to 24 GB, significantly less than the 40–80 GB available on current data center GPUs. This limits the maximum model size that can be trained on a single device and increases the risk of out-of-memory errors when multiple users share the same GPU through time-slicing. The heterogeneous VRAM distribution across nodes further complicates workload placement: a training job requiring 20 GB of GPU memory can only be scheduled on a subset of the cluster's nodes.

No high-speed interconnect. Enterprise training clusters typically use InfiniBand or RDMA over Converged Ethernet (RoCE) for low-latency, high-bandwidth communication between nodes during distributed training. The department's nodes are connected via standard Ethernet, introducing higher latency and lower bandwidth for inter-node communication. This constraint primarily affects large-scale distributed training workloads, which must be designed with higher communication-to-computation ratios in mind. The absence of InfiniBand also means that shared filesystem performance is limited by the network backbone, affecting data-intensive workloads such as computer vision pipelines.

No ECC memory. Consumer-grade GPUs and host systems lack Error-Correcting Code (ECC) memory, increasing the risk of silent data corruption during long-running training jobs. While this is a known limitation, its practical impact on typical academic ML workloads (training runs of hours to a few days) is relatively low compared to large-scale, week-long distributed training campaigns. The architecture does not attempt to mitigate this at the infrastructure level, but the use of experiment tracking (MLflow) and checkpointing strategies provides application-level resilience.

3.4.2 Advanced Scheduling Strategies

The absence of hardware-level GPU partitioning on consumer-grade cards means that multi-tenant GPU sharing must be orchestrated entirely in software. The architecture leverages Volcano, a Kubernetes batch scheduling plugin, as the primary mechanism for achieving fair, efficient, and predictable resource allocation.

Queue-based fair sharing. Volcano introduces the concept of *queues*, logical partitions of cluster resources with configurable weights, capacities, and priority thresholds.

Each user or research group can be assigned a dedicated queue with a guaranteed resource share. When the cluster is underutilized, queues can reclaim idle resources from other queues, ensuring high overall utilization without sacrificing fairness. This provides a software-level analogue to Slurm's fair-share scheduling policies, implemented entirely within the Kubernetes control plane.

Gang scheduling. Distributed training jobs require all participating pods to be scheduled simultaneously; partial placement leads to wasted resources as allocated GPUs sit idle waiting for the remaining pods. Volcano's gang scheduling semantics ensure that a job is either fully placed across the required nodes or not scheduled at all, with resources reserved until the entire gang can be accommodated. This is particularly important on a small cluster where the number of available GPUs is limited and partial allocation can block other jobs.

GPU time-slicing. On NVIDIA consumer GPUs, the NVIDIA driver supports a software-level time-slicing mechanism that allows multiple processes to share a single GPU by interleaving their execution across configurable time slices. The architecture configures time-slicing at the node level and exposes the resulting shared GPU resources through Kubernetes device plugins. While time-slicing does not provide memory isolation (all processes share the full VRAM address space), it enables concurrent lightweight workloads such as inference endpoints or short validation runs to coexist on the same GPU without requiring exclusive allocation.

Priority and preemption. Volcano supports hierarchical job priorities with configurable preemption policies. High-priority batch training jobs can preempt lower-priority interactive sessions (or vice versa, depending on the configured policy), ensuring that time-sensitive workloads are not indefinitely blocked by long-running jobs. The preemption strategy is configured to balance research flexibility (allowing interactive exploration) with throughput guarantees (ensuring batch training jobs complete within expected timeframes).

3.4.3 Elasticity and Dynamic Resource Management

The architecture supports dynamic scaling of both batch and interactive workloads through complementary mechanisms.

Ray cluster elasticity. Kube-Ray, the Kubernetes operator for Ray clusters, supports autoscaling of Ray worker nodes based on the resource demands of submitted tasks. When a distributed training job requires additional compute, Kube-Ray can

provision additional Ray workers (subject to available cluster resources) and automatically scale down when the workload completes. This elasticity is bounded by the fixed size of the physical cluster but provides flexible allocation within those limits.

Interactive workload hibernation. JupyterHub, configured as the primary interactive development environment, supports profile-based resource allocation. User notebooks can be configured with idle timeouts that automatically cull inactive sessions, freeing GPU resources for batch workloads during periods of low interactive demand (e.g., overnight). When a user reconnects, the notebook server is transparently restarted with the same persistent volume, preserving the user's data and environment.

Node-level resource overcommit. Given the small cluster size and the variable resource demands of ML workloads, the architecture supports controlled resource overcommit for CPU and memory (but not GPU VRAM). Best-effort workloads such as data preprocessing tasks or experiment visualization can be scheduled on nodes that are nominally at capacity, relying on Kubernetes' QoS class enforcement to evict them if higher-priority workloads require the resources. This strategy increases cluster utilization at the cost of occasional preemption, a trade-off that is acceptable in an academic research context where jobs are typically idempotent and can be restarted from checkpoints.

3.5 Summary

Concluding synthesis of modular design meets the problem statement.

Chapter 4

Implementation & Technical Realization

4.1 Deployment Strategy

The cluster is deployed using Ansible playbooks that automate the provisioning of K3s across all nodes. A single control-plane node is initialized as the K3s server, after which worker nodes are bootstrapped as agents and automatically joined to the cluster via a shared token. The Ansible playbooks handle host-level configuration such as package dependencies, kernel module loading (NVIDIA drivers, container runtime), user setup, and networking, as well as the K3s installation itself. This semi-manual approach keeps the bootstrapping process transparent and auditable while reducing the risk of configuration drift across nodes.

Once the K3s cluster is operational, all platform components: JupyterHub, Volcano, Kube-Ray, MLflow, Keycloak, Harbor, and the observability stack are installed and managed exclusively through Helm charts with environment-specific value overrides. Chart lifecycle management (installation, upgrades, rollbacks) is automated via GitHub Actions pipelines, which lint, template, and apply the Helm releases on each push to the main branch. This CI/CD-driven approach ensures that every change to the cluster configuration is version-controlled, reviewable, and reproducible, supporting the single-administrator operability goal: the platform can be reconstructed from the repository on a fresh K3s installation with no manual configuration steps beyond the initial Ansible bootstrap.

4.2 Implementing the Orchestration Engine

4.2.1 Configuring Volcano for Academic Constraints

Priority queues and gang-scheduling.

4.2.2 Integrating Kube-Ray for Distributed Scaling

Distributed compute scaling.

4.3 Building the Research Interface

How the API Gateway and JupyterHub provide the unified interface for the end-user.

4.4 Data and Lifecycle Automation

The integration of Harbor for image stability and MLflow for scientific experiment tracking.

4.5 Observability and Monitoring

Monitoring in this platform serves two distinct goals with different requirements. Operationally, it must convert failures from user-visible to administrator-visible: the incident analysed in Section 4.6.1 was detected through a failed user spawn precisely because no node-level GPU telemetry existed, and the alerting introduced since is designed to close that gap. Methodologically, time-series data on GPU utilization, scheduling behavior, and service health constitutes the empirical foundation for evaluating the resource-efficiency and operability claims of Chapter 3; without continuous measurement, such claims are anecdotal. Monitoring is therefore treated as a first-class platform component rather than an operational afterthought, following the practice standardized in large-scale production systems, where the four golden signals of latency, traffic, errors, and saturation form the core of service health assessment Beyer et al. [2016].

4.5.1 Stack Selection and Deployment

The metrics stack is built from the kube-prometheus-stack distribution of the Prometheus ecosystem. Prometheus [Volz and Rabenstein, 2015] descends from Google's Borgmon monitoring system used alongside the Borg cluster manager [Burns et al., 2016]; it consolidates a multi-dimensional time series database (TSDB), pull-based scraping with service discovery, and a dedicated query language into a single open-source system,

and has become the de facto metrics standard in the Kubernetes ecosystem. Three properties determined the choice. First, the Prometheus Operator extends the Kubernetes API with custom resources (`ServiceMonitor`, `PrometheusRule`), so that scrape targets and alerting rules are expressed declaratively and managed by the same Git-driven lifecycle as every other platform component. Second, the exporter ecosystem already covers the layers this platform must observe: `node-exporter` for host metrics, `kube-state-metrics` for the state of Kubernetes objects, DCGM for GPU telemetry, and native `/metrics` endpoints of JupyterHub, SeaweedFS, and Traefik. Third, the stack is sufficiently compact to fit the resource envelope of the service node, consistent with the resource-efficiency objective of Chapter 3.

Grafana fronts the stack for dashboards and is integrated into the Keycloak identity layer like every other user-facing service, so that access to operational data requires no separate credentials. Alertmanager receives firing alerts from Prometheus rule evaluation; alert delivery is currently limited to the dashboards themselves, which matches the single-administrator scale of the platform, and can be extended to email or chat webhooks without structural changes.

4.5.2 Tiered Metric Storage

The storage philosophy layers media by access pattern and failure semantics rather than using a single backend. The Prometheus write-ahead log (WAL) and TSDB head require a local filesystem with crash-safe writes: a network filesystem mounted through FUSE, such as the SeaweedFS CSI storage class used for notebook volumes, cannot guarantee WAL integrity across mount interruptions, and documented production incidents show WAL corruption and query failures when the metrics database itself resides on such a mount. The hot tier therefore runs on node-local storage with a 30-day retention window, which covers routine operational lookback.

Long-term retention instead uses the SeaweedFS S3 object interface, its strongest and most native access mode. A Thanos sidecar [tha] ships compacted two-hour blocks from the local TSDB to a dedicated `monitoring` bucket, where they accumulate for the duration of the platform's lifetime; this is the data source for the utilization analyses of the evaluation, surviving local rotation and node rebuilds. The layering preserves a deliberate independence property: the monitoring system must not depend on the infrastructure it monitors. If SeaweedFS is unavailable, block uploads retry while the local buffer absorbs the delay, and no scrape path traverses the storage layer. The same pattern, local hot state with SeaweedFS S3 for durable bulk data, mirrors the platform-wide data lifecycle decisions for registry images and experiment artifacts, reusing one storage system through the interface it provides best.

4.5.3 GPU Telemetry and Driver-Health Alerting

GPU nodes run the NVIDIA DCGM exporter [NVIDIA Corporation], which exposes per-GPU counters (utilization, memory occupancy, temperatures, ECC and XID error events) through the standard `/metrics` endpoint. Deployment requires the NVIDIA container runtime to inject the host driver libraries and device nodes into the exporter pod; without this injection the exporter starts and serves metrics with an empty result, a failure mode that is silent to liveness probes. The heterogeneous GPU fleet constrains the exporter version: current DCGM releases drop support for the Pascal architecture of the TITAN Xp, so the deployed 3.3 series is retained to keep both GPU models fully observable.

The primary alert, `DCGMExporterDown`, fires on exporter liveness rather than on individual counters. This design targets precisely the incident class of Section 4.6.1: a broken host driver prevents NVML initialization, the exporter crash-loops, and the alert fires independently of whether any GPU workload is running, converting the outage from user-visible to administrator-visible. A complementary rule alerts on XID error events, which indicate hardware and driver-level faults. In the division of labor argued in Section 4.6.2, these alerts act as the surfacing mechanism for both human and AI operators: they shorten the detection phase, which the incident analysis identified as the phase most improved by tooling rather than by further automation of actions.

4.6 Operational Experience

A platform operated by a single administrator is exercised primarily by incidents: failures that must be detected, diagnosed, and remediated without a dedicated operations team. This section documents one representative production incident as a case study of the operability objectives formulated in Chapter 3. The incident is instructive because it originates below the Kubernetes abstraction, in host-level package management on a worker node, yet manifests as a user-facing workload failure. It therefore delimits what container orchestration does and does not isolate.

4.6.1 Case Study: GPU Driver Version Mismatch

On 22 August 2026, a user-requested GPU notebook crash-looped at container creation. The pod event stream contained the decisive error, `nvidia-container-cli: initialization error: nvml error: driver/library version mismatch`, in the NVIDIA runtime's pre-start hook. Detection was by user impact rather than by the platform: the cluster had no node-level GPU health monitoring at the time, so no alert preceded the failed spawn. The event stream alone sufficed to localize the fault to the

GPU stack of one worker node and to exclude the scheduler, the storage layer, and the notebook image.

Running `nvidia-smi` on the node produced the identical error, ruling out the container layer. The node, up for 38 days, was running kernel 6.8.0-134 with the NVIDIA kernel module 580.159.03 loaded at boot, whereas Ubuntu's unattended-upgrades had silently updated the userspace driver libraries to 580.173.02 on the previous morning and installed a new kernel (6.8.0-138) with matching precompiled modules. NVML refuses to operate across this version skew, so every subsequent GPU container creation on the node failed. A module reload could not have restored service: the on-disk module for the running kernel was still 580.159.03, and a matching module existed only for the not-yet-booted kernel. A reboot into the new kernel was therefore the minimal correct remediation.

After the reboot, the kernel module and the userspace libraries agreed at 580.173.02, and both GPUs (an NVIDIA TITAN V and a TITAN Xp) reported healthy. One secondary fault remained: the notebook pod scheduled during the reboot window was stuck in a terminal `UnexpectedAdmissionError` state, rejected with "Allocate failed due to no healthy devices present; cannot allocate unhealthy devices `volcano.sh/vgpu-memory`". This was a stale admission decision made before the vGPU device plugin had re-registered its devices with the kubelet; deleting the pod allowed JupyterHub to spawn a healthy replacement. User-visible recovery from the first failed spawn took approximately seven minutes.

Two preventive measures follow. First, GPU driver upgrades must be coupled to a maintenance action: either reboot promptly after a driver upgrade, or hold `nvidia-*` and `linux-modules-nvidia-*` back from unattended-upgrades and upgrade only during planned windows. The version-skew window is otherwise unbounded; in this case it lasted 26 hours. Second, node-level GPU health checks, such as a periodic `nvidia-smi` probe or a DCGM exporter per GPU node, would convert such outages from user-visible to administrator-visible and shrink detection time independently of workload activity.

4.6.2 Agentic AI in the Operations Loop

This incident was diagnosed and remediated with the assistance of an agentic AI assistant granted tool access to the cluster: a read-only Kubernetes API and SSH access to nodes. The assistant autonomously executed the complete diagnostic chain, from pod event triage and node inspection through kernel-module and library version comparison, package-manager history, and a cross-kernel `modinfo` check that established the insufficiency of a module reload. It then narrowed the response space to a small set of remediation options with explicit trade-offs: drain-and-reboot, plain reboot, or

deferral to a maintenance window. The administrator's entire contribution was a single executive decision among these options, which the assistant executed and verified, including the resolution of the secondary admission failure and a post-recovery GPU smoke test.

This division of labor, with machine-executed diagnostics and human-owned judgment and accountability, directly supports the single-administrator operability objective of Chapter 3: it reduces the expertise and wall-clock time required per incident while keeping the administrator in the loop at the point where responsibility lies. The observed bottleneck was decision latency rather than diagnostic skill, which suggests that the marginal investment is not further automation of actions but better surfacing of options; the observability stack of Section 4.5 operationalizes exactly this conclusion.

Bibliography

Thanos: Highly available prometheus setup with long term storage capabilities. `https://thanos.io`. Accessed 2026-08-22.

Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. ISBN 978-1491929124.

Brendan Burns, Brian Grant, David Oppenheimer, Eric A. Brewer, and John Wilkes. Borg, omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.

Rajkumar Buyya, Satish Narayana Srirama, Giuliano Casale, Rodrigo N. Calheiros, Yogesh Simmhan, Bipoš Varghese, Sherali Karaulov, Adel Nadjaran Toosi, Marcos D. Assunção, Saurabh Kumar Garg, Raksha M. Vahid, Mariam Kiran, Emmanuele Santos, Karoč Wilczyński, George Kousiouris, Jia Ruan, Christian Baun, Lars Kolbe, Christian Pichler, Bernhard Kolb, Jörg Domaschka, and Stefan Wesner. A manifesto for future generation cloud computing: Research directions for the next decade. *ACM Computing Surveys*, 51(5):1–38, 2018. doi: 10.1145/3241737.

Antony Chazapis, Fotis Nikolaidis, Manolis Marazakis, and Angelos Bilas. Running Kubernetes workloads on HPC. In *ISC High Performance Workshops*. Springer, 2023. doi: 10.1007/978-3-031-47187-0_12.

Jonathan Decker and Julian Kunkel. Ephemeral Kubernetes: Dynamically deleting and recreating clusters using Warewulf. *Journal of Supercomputing*, 81(16), 2025. doi: 10.1007/s11227-025-07668-y.

John Domingus, Arun Lakshmanan, Justin Armitage, and Andrew C. Munro. *Cloud Native DevOps with Kubernetes*. O'Reilly Media, 2nd edition, 2022.

Paolo Di Francesco, Patricia Lago, and Ivano Malavolta. Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*, 150:77–97, 2019. doi: 10.1016/j.jss.2019.01.001.

Víctor Medel Gracia, Rafael Tolosana-Calasanz, José Ángel Bañares, and Omer F. Rana. Characterising resource management performance in Kubernetes. In *Computers & Electrical Engineering*, volume 68, pages 261–280. Elsevier, 2018. doi: 10.1016/j.compeleceng.2018.03.041.

Kelsey Hightower, Brendan Burns, and Joe Beda. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media, 2nd edition, 2019.

Pooyan Jamshidi, Claus Pahl, Nabor C. Mendonça, James Lewis, and Stefan Tilkov. Microservices: The journey so far and challenges ahead. *IEEE Software*, 35(3):24–35, 2018. doi: 10.1109/MS.2018.2141039.

Claudia Misale, Daniel J. Milroy, Carlos Eduardo Arango Gutierrez, Maurizio Drocco, Stephen Herbein, Dong H. Ahn, Zvonko Kaiser, and Yoonho Park. Towards standard Kubernetes scheduling interfaces for converged computing. In *Sustained Simulation Performance*, pages 229–245. Springer, 2022. doi: 10.1007/978-3-030-96498-6_18.

Thanh-Tung Nguyen, Yu-Jin Yeom, Taehong Kim, Dae-Heon Park, and Sehan Kim. Horizontal pod autoscaling in Kubernetes for elastic container orchestration. *Sensors*, 20(16):4621, 2020. doi: 10.3390/s20164621.

NVIDIA Corporation. Dcgm exporter: Nvidia gpu metrics exporter for prometheus. `https://github.com/NVIDIA/dcgm-exporter`. Accessed 2026-08-22.

D.A. Petrosyan. Scheduling ML and HPC jobs with Shoc platform over Kubernetes. *Physics of Particles and Nuclei*, 56:1581–1585, 2025.

Mauricio Salatino. *Platform Engineering on Kubernetes*. Manning Publications, 2024.

Khaldoun Senjab, Sohail Abbas, Naveed Ahmed, and Atta ur Rehman Khan. A survey of Kubernetes scheduling algorithms. *Journal of Cloud Computing*, 12:1–26, 2023. doi: 10.1186/s13677-023-00471-1.

Manolis Skoularikis, Athanasios Liatifis, Dimitrios Pliatsios, Vasileios Argyriou, Evangelos K. Markakis, Thomas D. Lagkas, Georgios Th. Papadopoulos, and Panagiotis Sargiannidis. Kubernetes in edge and cloud computing: A comparative study of K3s, K0s, MicroK8s, and K8s. In *International Conference in Electronic Engineering and Information Technology (EEITE)*. IEEE, 2025. doi: 10.1109/EEITE65381.2025.11166028.

Jacopo Soldani, Damian Andrew Tamburri, and Willem-Jan van den Heuvel. The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software*, 143:151–178, 2018. doi: 10.1016/j.jss.2018.09.082.

Viktória Spišaková, Dalibor Klusáček, and Lukáš Hejtmánek. Using Kubernetes in academic environment: Problems and approaches. In *Job Scheduling Strategies for Parallel Processing (JSSPP)*, pages 202–222. Springer, 2023. doi: 10.1007/978-3-031-22698-4_12.

Sari Sultan, Imtiaz Ahmad, and Tassos Dimitriou. Container security: Issues, challenges, and the road ahead. *IEEE Access*, 7:52976–52996, 2019. doi: 10.1109/ACCESS.2019.2911732.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, pages 1–17. ACM, 2015. doi: 10.1145/2741948.2741964.

Julius Volz and Björn Rabenstein. Prometheus: A next-generation monitoring system. In *SREcon15 Europe*, Dublin, Ireland, 2015. USENIX Association. URL `https://www.usenix.org/conference/srecon15europe/program/presentation/rabenstein`.

Tim Wickberg. Slinky: Slurm in Kubernetes — performant AI and HPC workload management. In *KubeCon Europe*, 2025.

Diyaz Yakubov and David Hästbacka. Comparative analysis of lightweight kubernetes distributions for edge computing: Performance and resource efficiency. In *European Conference on Service-Oriented and Cloud Computing (ESOCC)*. Springer, 2025a. doi: 10.1007/978-3-031-84617-5_7.

Diyaz Yakubov and David Hästbacka. Comparative analysis of lightweight kubernetes distributions for edge computing: Security, resilience and maintainability. In *European Conference on Service-Oriented and Cloud Computing (ESOCC)*. Springer, 2025b. doi: 10.1007/978-3-031-84617-5_8.

Andy B. Yoo, Morris A. Jette, and Mark Grondona. SLURM: Simple Linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing (JSSPP)*, pages 44–60. Springer, 2003. doi: 10.1007/10968987_3.

Zhiheng Zhong and Rajkumar Buyya. A cost-efficient container orchestration strategy in Kubernetes-based cloud computing infrastructures with heterogeneous resources. *ACM Transactions on Internet Technology*, 20(2):1–24, 2020. doi: 10.1145/3378447.